UC2

Tratamento e Auditoria de Dados

Base: NaraGroup RH
funcionarios_naragroup_dirty.csv

Prof. Cassio Ribeiro
SENAC RJ

8 tipos de problema. Um pipeline de solução.

01 Nulos	02 Duplicatas	03 Tipo errado	04 Datas
05 Categorias	06 Espaços	07 Impossíveis	08 Outliers

Por que os dados chegam sujos?

No mundo real, bases de dados vêm de sistemas legados, formulários manuais, integrações entre sistemas diferentes e exportações automáticas. Nenhuma base chega pronta para análise.

Sistemas legados

Softwares antigos que não validam dados na entrada — aceita qualquer coisa que o usuário digitar.

Preenchimento manual

Formulários físicos digitados por pessoas — erros de digitação, campos em branco, padrões inconsistentes.

Integração entre sistemas

Cada sistema tem seu formato de data, separador decimal e nomenclatura — ao juntar, os conflitos aparecem.

Exportações automáticas

Scripts de exportação que não verificam qualidade — duplicam registros, truncam campos, perdem encoding.

O primeiro olhar: entendendo a base

Antes de qualquer tratamento, explore a base. O objetivo é entender o tamanho, os tipos de cada coluna e identificar inconsistências visíveis.

```
df = pd.read_csv('funcionarios_naragroup_sujo.csv', sep=';')

print(df.shape)     # (linhas, colunas)
display(df.head())    # primeiras linhas
display(df.dtypes)    # tipo de cada coluna
df.info()                # resumo completo — nulos inclusos
```

shape

Quantas linhas e colunas chegaram

dtypes

Colunas numéricas virando object é sinal de problema

head()

Visualização rápida — inconsistências já aparecem

info()

Mostra nulos e tipos numa só chamada

Problema 1 — Valores nulos

Como identificar

```
df.isnull().sum()          # conta nulos por coluna
```

Três estratégias — escolha depende do contexto e do volume de nulos:

1 Remover linhas

```
df.dropna()
df.dropna(subset=['salario'])
```

Use quando o nulo é minoria e a linha não tem valor analítico.

2 Preencher com valor fixo

```
df['email'].fillna('sem_email')
df['salario'].fillna(0)
```

Use quando há um valor padrão lógico para o contexto.

3 Preencher com a média

```
media = df['salario'].mean()
df['salario'].fillna(media)
```

Use em variáveis numéricas quando remover causaria perda relevante.

Problema 2 — Linhas duplicadas

Mesmo registro aparecendo mais de uma vez — pode ser duplicata total (linha idêntica) ou parcial (mesmo funcionário, IDs diferentes).

```
# Verificar quantas duplicatas existem
df.duplicated().sum()

# Ver quais são as linhas duplicadas
df[df.duplicated()]

# Remover duplicatas — manter a primeira ocorrência
df = df.drop_duplicates()

# Remover duplicatas por colunas específicas
df = df.drop_duplicates(subset=['nome','departamento'])
```

`duplicated()`

Retorna True para cada linha que é duplicata. .sum() conta quantas.

`drop_duplicates()`

Remove as duplicatas, mantendo a primeira ocorrência por padrão.

`subset=['col']`

Identifica duplicatas só por colunas específicas — útil quando o ID é diferente mas o conteúdo é igual.

Problema 3 — Tipo de dado errado

Coluna numérica que chegou como texto — causa mais comum: separador decimal errado ou unidade junto ao número.

Como chega na base

```
"3.500,00"   "12.800,50"   "850,00"
```

Como deve ficar

```
3500.0       12800.5       850.0
```

```python
# Passo 1: trocar vírgula por ponto e remover caracteres extras
df['salario'] = df['salario'].str.replace(',', '.')

# Passo 2: converter para numérico
df['salario'] = pd.to_numeric(df['salario'], errors='coerce')
```

errors='coerce' → valores que não conseguirem ser convertidos viram NaN automaticamente. Nunca quebra o código.

Problema 4 — Datas inconsistentes

Três formatos diferentes na mesma coluna — situação muito comum em exportações manuais e integrações entre sistemas.

2021-03-15

ISO 8601

15/03/2021

Brasileiro

Mar-2021

Abreviado

```
# pd.to_datetime lida com múltiplos formatos automaticamente
df['data_admissao'] = pd.to_datetime(df['data_admissao'],
                                     errors='coerce', dayfirst=True)

# Verificar datas que não conseguiram ser convertidas (NaT)
df[df['data_admissao'].isnull()]
```

`errors='coerce'`

Datas inválidas viram NaT (Not a Time) — fácil de identificar depois com isnull().

`dayfirst=True`

Indica que o formato brasileiro (dia/mês/ano) deve ser interpretado corretamente.

Problema 5 — Categorias sujas

Mesmo valor escrito de formas diferentes — causam problemas silenciosos em groupby e filtros.

Mesma categoria, 5 formas diferentes: "ti" · "TI" · "T.I." · "Tecnologia" · "tecnologia da informação"

```python
# Passo 1: remover espaços e padronizar para maiúsculo
df['departamento'] = df['departamento'].str.strip().str.upper()

# Passo 2: ver quais valores únicos restaram
df['departamento'].unique()

# Passo 3: mapear variações para o valor correto
mapa = {
    'T.I.': 'TI', 'TECNOLOGIA': 'TI', 'TECNOLOGIA DA INFORMAÇÃO': 'TI',
    'R.H.': 'RH', 'RECURSOS HUMANOS': 'RH',
}
df['departamento'] = df['departamento'].map(mapa).fillna(df['departamento'])
```

fillna(df['departamento']) → mantém os valores que já estavam corretos e não precisavam de remapeamento.

Problema 6 — Espaços invisíveis

Strings com espaços no início ou fim parecem normais na tela — mas causam falhas silenciosas em filtros e agrupamentos.

O problema silencioso:

```
df[df['nome'] == 'Ana Lima']          # retorna 0 linhas — o espaço estraga o filtro
```

```
# Solução: aplicar str.strip() em todas as colunas de texto
colunas_texto = ['nome', 'departamento', 'email']

for col in colunas_texto:
    df[col] = df[col].str.strip()
```

Boa prática: aplicar str.strip() logo no início do tratamento, antes de qualquer filtro ou groupby.

Problema 7 — Valores impossíveis

Dados que passaram pela validação do sistema mas não fazem sentido no mundo real — salário negativo, idade 200 anos, distância zero.

```python
# Identificar salários negativos ou zerados
df[df['salario'] < 0]

# Remover registros com valores impossíveis
df = df[df['salario'] > 0]

# Filtrar dentro de um intervalo válido
df = df[df['salario'].between(1000, 50000)]

# Contar quantos registros foram removidos
print(f'Registros removidos: {antes - len(df)}')
```

> Sempre registre quantos registros foram removidos em cada etapa — é parte da auditoria e deve ser documentado.

Problema 8 — Outliers extremos

Valores absurdos que passaram pela validação — erros de digitação como R$ 999.999 num salário. Retomada do IQR da Aula 04.

```
array_sal = np.array(df['salario'].dropna())

q1  = np.percentile(array_sal, 25)
q3  = np.percentile(array_sal, 75)
iqr = q3 - q1
lim_sup = q3 + (1.5 * iqr)

# Remover salários acima do limite
df = df[df['salario'] <= lim_sup]

print(f'Limite superior: {lim_sup:.2f}')
```

O mesmo raciocínio se aplica ao limite inferior — salários negativos ou abaixo de um mínimo esperado também são outliers.

Fluxo completo de auditoria

Pipeline recomendado — nessa ordem:

1 Explorar

`shape · dtypes · info()`

2 Nulos

`isnull() · dropna() · fillna()`

3 Duplicatas

`duplicated() · drop_duplicates()`

4 Tipos

`pd.to_numeric() · pd.to_datetime()`

5 Categorias

`str.strip() · str.upper() · map()`

6 Espaços

`str.strip() em todas as colunas texto`

7 Impossíveis

`filtro com .loc · between()`

8 Outliers

`IQR · limites superior e inferior`

Atividade Prática — NaraLog

A NaraLog exportou sua base de entregas do sistema legado para migração. O gerente de operações precisa da base limpa para iniciar as análises. O arquivo entregas_naralog_sujo.csv chegou com diversos problemas — audite e trate antes de qualquer análise.

01 — Explore a base

Quantas linhas e colunas? Quais colunas têm nulos e quantos? Quais colunas têm tipo incorreto?

02 — Trate os nulos

Trate distancia_km e valor_frete. Justifique: remover ou preencher?

03 — Remova duplicatas

Identifique e remova as entregas duplicadas.

04 — Corrija os tipos

distancia_km chegou como texto. Converta para numérico.

05 — Padronize o status

A coluna status tem o mesmo valor escrito de muitas formas diferentes.

06 — Espaços e impossíveis

Trate espaços invisíveis nas colunas de texto e remova registros com distancia_km = 0 e peso negativo.

07 — Outliers em valor_frete

Use o IQR para identificar e remover valores absurdos.

08 — Resultado final

Quantas linhas sobraram? Compare com o shape inicial e exiba a base limpa.

O que vimos hoje

1. `isnull() · dropna() · fillna()`
   Identificar e tratar valores nulos

2. `duplicated() · drop_duplicates()`
   Identificar e remover duplicatas

3. `pd.to_numeric(errors='coerce')`
   Converter tipos errados sem quebrar o código

4. `pd.to_datetime(errors='coerce')`
   Unificar formatos de data inconsistentes

5. `str.strip() · str.upper() · map()`
   Padronizar categorias sujas

6. `str.strip() em colunas de texto`
   Eliminar espaços invisíveis

7. `filtro com .loc · between()`
   Remover valores impossíveis

8. `IQR — q1, q3, iqr, limite_sup`
   Identificar e remover outliers extremos